CoderCommands MCP System

Step 11 Record — Postgres MCP Gateway Endpoint

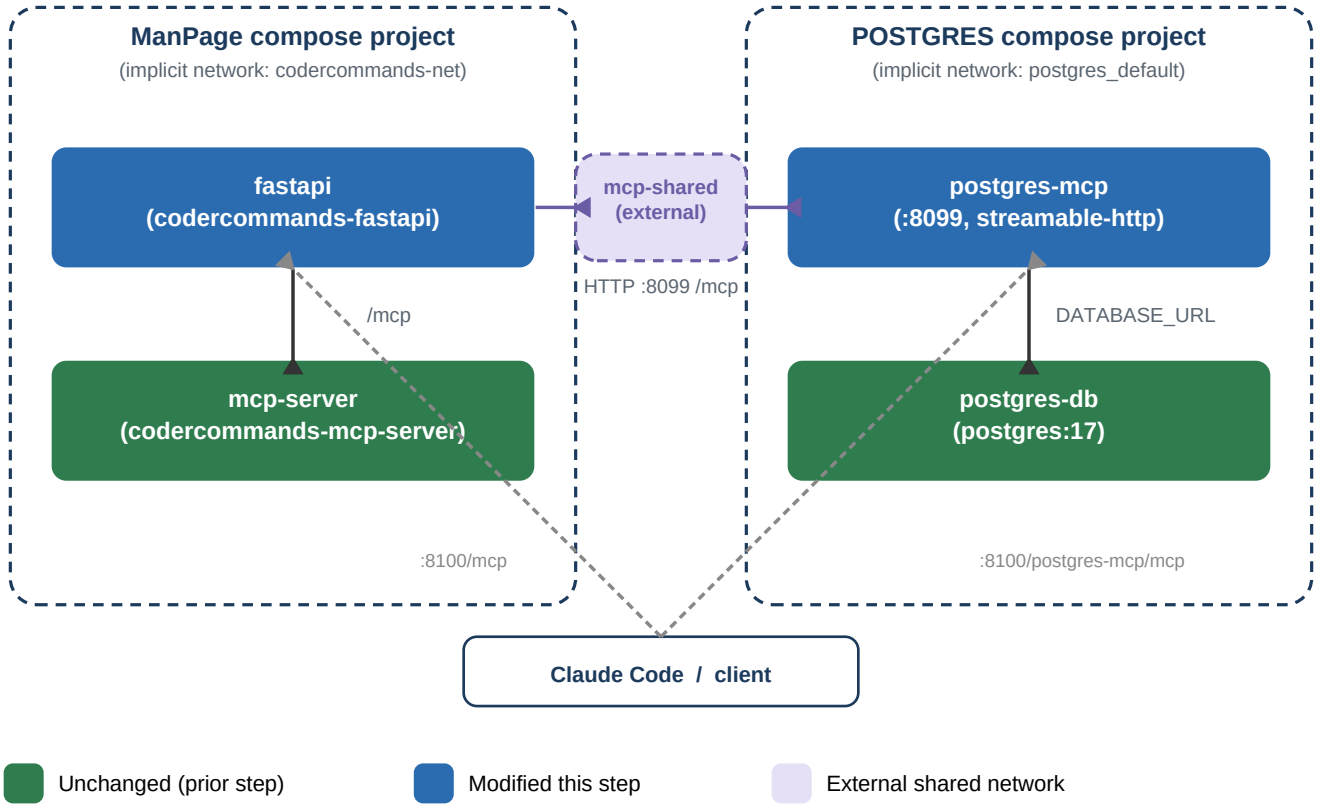
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	11 — Postgres MCP Gateway Endpoint (post-plan addendum)
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Add a second proxied MCP endpoint to the FastAPI gateway so Claude Code can reach **postgres-mcp** -- the Model Context Protocol server that exposes the application Postgres database at C:\Users\katlego.gagoopane\Documents\2026\POSTGRES -- the same way it already reaches this project's own **mcp-server**. postgres-mcp runs as a container in a wholly separate docker compose project with its own compose file, its own default network, and its own lifecycle. This record covers the new route, the cross-project Docker networking that makes it reachable, and the trade-offs of that networking choice versus folding everything into one compose file.

2. System Change Diagram

Both compose projects, their internal (default) networks, and the new external mcp-shared network that bridges them. fastapi and postgres-mcp are each attached to two networks simultaneously: their own project's default network, and mcp-shared.



3. File / Component Register

Every file touched to add this endpoint, with its role and interfaces (I/O-list style, matching prior step records).

Ref	File / Component	Role	Interface In	Interface Out
R1	api/main.py	Adds a second httpx.AsyncClient plus a proxy_postgres_mcp route mounted at /postgres-mcp{path:path}, mirroring the existing /mcp proxy (headers/body/stream forwarded as-is) but stripping the gateway-only /postgres-mcp prefix before forwarding.	HTTP requests to /postgres-mcp/* from clients	HTTP to postgres-mcp upstream (http://postgres-mcp:8099/mcp*)
R2	docker-compose.yml (ManPage project)	fastapi service joins new external mcp-shared network and gets POSTGRES_MCP_UPSTREAM_URL; top-level networks: block declares mcp-shared as external.	mcp-shared network (must pre-exist on host)	fastapi container attached to 2 networks
R3	../POSTGRES/compose.yaml	postgres-mcp service now lists networks: [default, mcp-shared] explicitly; top-level networks: block declares mcp-shared as external.	mcp-shared network (must pre-exist on host)	postgres-mcp container attached to 2 networks

Ref	File / Component	Role	Interface In	Interface Out
R4	.env.example	Documents the postgres-mcp upstream and the one-time docker network create mcp-shared prerequisite (comment only -- no new .env variable, URL is set directly in compose).	—	Whoever configures a fresh checkout
R5	README.md	Adds postgres-mcp to the service table, documents the network prerequisite and bringing up both projects, and adds a second claude mcp add registration command.	—	Whoever operates or extends this system next

4. Work Performed

• Added a second httpx.AsyncClient in api/main.py (_pg_client), configured from a new POSTGRES_MCP_UPSTREAM_URL environment variable (default http://postgres-mcp:8099), managed in the same lifespan context manager as the existing MCP client.

• Added a proxy_postgres_mcp route at /postgres-mcp{path:path} that duplicates the existing proxy_mcp function's behaviour exactly -- same hop-by-hop header stripping, same streamed request/response body, same StreamingResponse plumbing -- rather than generalising both routes behind a shared helper, matching this file's existing preference for explicit, unabstracted code over premature reuse for what is still just two cases.

• The new route strips the gateway-only /postgres-mcp prefix before forwarding, since postgres-mcp itself serves at /mcp -- a client calling http://gateway:8100/postgres-mcp/mcp reaches postgres-mcp's own /mcp path unchanged.

• Declared a new external Docker network, mcp-shared, in both docker-compose.yml (this project) and ../POSTGRES/compose.yaml, and joined it from the two services that need to talk across the project boundary: fastapi and postgres-mcp.

• Because Compose only auto-attaches a service to the implicit default network when that service has *no* explicit networks: key, postgres-mcp's compose entry now lists **both** default and mcp-shared explicitly -- omitting default would have silently cut it off from postgres-db.

• Updated .env.example and README.md: documented the new upstream, the one-time docker network create mcp-shared prerequisite, bringing up both compose projects, and a second claude mcp add registration command for the postgres-mcp server.

5. Cross-Compose Networking, Explained

Default isolation. Each docker compose project gets its own auto-created network (named <project>_default) and every service without an explicit networks: key joins it implicitly. Docker's embedded DNS server (127.0.0.11 inside every container) resolves other services on that same network by service name -- this is how fastapi already resolved mcp-server before this change. Two separate compose projects get two separate networks with **no route between them** by default; that isolation is a deliberate Compose feature, not an oversight.

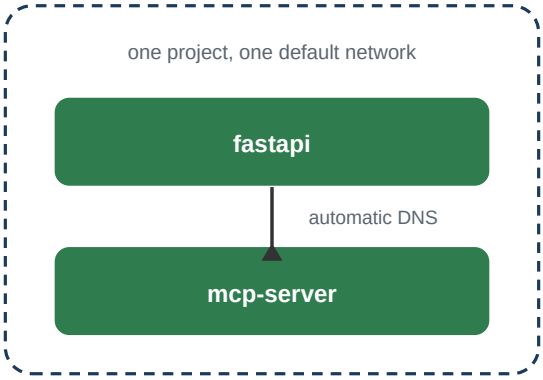
Bridging them. To let containers from different projects reach each other, a network is created *outside* either project's lifecycle: docker network create mcp-shared. This is an ordinary user-defined bridge network as far as the Docker Engine is concerned -- nothing about it belongs to either compose.yaml.

Opting in. Each compose file then declares that network under its top-level networks: key with external: true, which tells Compose "look this network up by name, don't try to create or manage it." Any service that lists mcp-shared in its own networks: key becomes a container attached to two networks at once, with one virtual NIC and one DNS scope per network -- so postgres-mcp still resolves postgres-db (default network) while also being resolvable as postgres-mcp by fastapi (mcp-shared network).

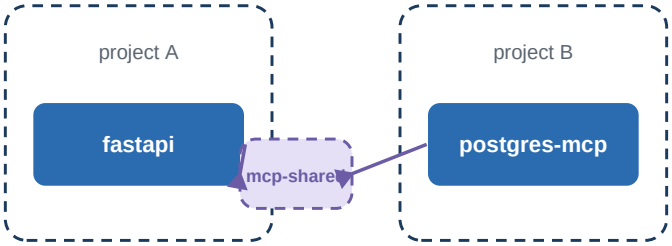
Lifecycle caveat. Because the network is external, whichever project comes up first does *not* create it -- it must already exist, or docker compose up fails with a "network not found" error. Neither project owns its lifecycle: docker compose down in either one will not remove it (Compose only removes networks it created itself), so cleanup is a manual docker network rm mcp-shared once both stacks are stopped.

Same idea, contrasted with the simpler single-project case:

Single compose file



Two compose files



manual: docker network create
explicit: networks: [mcp-shared]

6. Pros & Cons: Same Compose File vs. Separate Compose Files

This project already runs four services in one compose file with automatic networking; postgres-mcp could in principle have been added there too. It wasn't, because it's a pre-existing stack with its own compose file, its own volume, and its own reasons to be restarted independently. The table below is the general trade-off, not specific to this one endpoint.

	Single compose file (one project)	Two compose files (shared external network)
Startup ordering	depends_on + healthcheck give real, enforced ordering guarantees.	No cross-project ordering. fastapi can start before postgres-mcp is healthy; first proxied request may fail/retry.
DNS / networking	Automatic default network, zero extra steps, always works.	Requires a manual, out-of-band docker network create mcp-shared before either project; easy to forget.
Lifecycle coupling	Restarting/rebuilding any service touches the whole project.	Each stack starts, stops, and rebuilds independently -- rebuilding fastapi never touches the running postgres-mcp/postgres-db containers.
Ownership / reuse	Hard to have one service maintained separately or reused by other projects without copy-pasting its definition.	postgres-mcp can be owned, versioned, and reused by other consumer projects without duplication.
Cleanup	docker compose down [-v] fully tears down everything it created, including the network.	docker compose down never removes an external network or the other project's volumes; stale networks can accumulate if not docker network rm'd manually.
Blast radius	docker compose down -v wipes every volume in the project, including ones you may not have meant to touch.	A bad command in one project cannot touch the other project's containers or volumes at all.
Reachability surface	Implicitly scoped to exactly the services declared in this one file.	Any other project that also joins mcp-shared can reach postgres-mcp too -- the shared network widens reachability beyond either file alone, so it must be a deliberate choice.

7. Issues Encountered & Resolution

Initial draft of ../POSTGRES/compose.yaml's postgres-mcp service listed only mcp-shared under networks:. Adding any explicit networks: key removes a service's automatic membership in the project's implicit default network -- this would have silently broken postgres-mcp's connection to postgres-db (reached over DATABASE_URL). Resolved by listing both default and mcp-shared explicitly.

8. Verification Record

Check	Method	Result
New /postgres-mcp{path:path} route mirrors the existing /mcp proxy's header-stripping and streaming behaviour	Code review against api/main.py's existing proxy_mcp implementation	**PASS**

Check	Method	Result
mcp-shared is declared external: true in both compose files, so neither project tries to create or own it	Read back docker-compose.yml and ../POSTGRES/compose.yaml after edit	**PASS**
postgres-mcp keeps reachability to postgres-db after gaining an explicit networks: list	Confirmed default is listed alongside mcp-shared (explicit networks: opts a service out of implicit default-network membership)	**PASS**
End-to-end call through http://127.0.0.1:8100/postgres-mcp/mcp against a running postgres-mcp	Not yet executed this session -- both compose projects were not brought up	**PENDING**

9. Next Step / Open Dependencies

Run docker network create mcp-shared once, bring up both compose projects (docker compose up -d --build here, and the equivalent in ../POSTGRES), and confirm the end-to-end round trip through http://127.0.0.1:8100/postgres-mcp/mcp against a live postgres-mcp -- the one item still marked PENDING in the verification record above. After that, register it with claude mcp add --transport http postgres http://127.0.0.1:8100/postgres-mcp/mcp as documented in README.md.